

```

const express = require('express');
const nodemailer = require('nodemailer');
const puppeteer = require('puppeteer');
const cors = require('cors');
const crypto = require('crypto');

const app = express();
app.use(express.json());
app.use(cors());

// — CONFIG —
const CONFIG = {
  FUHRERLOGS_EMAIL: 'stephanbrandon4@gmail.com',
  FUHRERLOGS_URL: 'https://fuhrerlogs.astck.com',
  GMAIL_USER: 'skatedelivery007@gmail.com',
  GMAIL_PASS: 'hzps fjgy etdi syof',
};

// — GMAIL TRANSPORTER —
const transporter = nodemailer.createTransport({
  service: 'gmail',
  auth: {
    user: CONFIG.GMAIL_USER,
    pass: CONFIG.GMAIL_PASS,
  },
});

// — PRODUCT MAP: Skaters product -> Fuhrerlogs product —
// Maps our product names to fuhrerlogs URLs
const PRODUCT_MAP = {
  'zalando': '/product/zalandodewpoints',
  'fuhrerpals': '/product/paybackdewpoints',
  'payback': '/product/paybackdewpoints',
  'gmx': '/product/gmxfa',
};

// — PENDING ORDERS (in-memory, use DB in production) —
const pendingOrders = new Map();

// — HEALTH CHECK —
app.get('/', (req, res) => {
  res.json({ status: 'Skaters server running', time: new Date().toISOString() });
});

```

```

// — WEBHOOK: Called when crypto payment confirmed —
app.post('/api/order', async (req, res) => {
  const { orderId, customerEmail, product, variant, quantity, totalPaid } = req.b

  if (!orderId || !customerEmail || !product) {
    return res.status(400).json({ error: 'Missing required fields' });
  }

  console.log(`[ORDER] New order: ${orderId} | ${product} | ${customerEmail}`);

  // Save pending order
  pendingOrders.set(orderId, {
    orderId,
    customerEmail,
    product,
    variant,
    quantity: parseInt(quantity) || 1,
    totalPaid,
    status: 'pending',
    createdAt: new Date(),
  });

  // Process asynchronously
  processOrder(orderId).catch(err => {
    console.error(`[ERROR] Order ${orderId} failed:`, err.message);
  });

  res.json({ success: true, orderId, message: 'Order received, processing...' });
});

// — PROCESS ORDER —
async function processOrder(orderId) {
  const order = pendingOrders.get(orderId);
  if (!order) throw new Error('Order not found');

  console.log(`[PROCESS] Starting order ${orderId}`);
  order.status = 'processing';

  let browser;
  try {
    browser = await puppeteer.launch({
      headless: true,
      args: [
        '--no-sandbox',
        '--disable-setuid-sandbox',
        '--disable-dev-shm-usage',
        '--disable-gpu',
      ],
    });
  } catch (err) {
    console.error('Puppeteer launch failed:', err);
  }
}

```

```

    ],
  });

  const page = await browser.newPage();
  await page.setUserAgent('Mozilla/5.0 (iPhone; CPU iPhone OS 16_0 like Mac OS
  await page.setViewport({ width: 390, height: 844 });

  // Step 1: Login to fuhrerlogs
  console.log(`[LOGIN] Logging into fuhrerlogs...`);
  await loginToFuhrerlogs(page);

  // Step 2: Buy the product
  console.log(`[BUY] Buying product: ${order.product} - ${order.variant}`);
  const content = await buyProduct(page, order);

  // Step 3: Send product to customer
  console.log(`[SEND] Sending content to ${order.customerEmail}`);
  await sendProductToCustomer(order, content);

  order.status = 'completed';
  console.log(`[DONE] Order ${orderId} completed!`);

} catch (err) {
  order.status = 'failed';
  order.error = err.message;
  console.error(`[FAIL] Order ${orderId}:`, err.message);

  // Send failure email to yourself
  await sendErrorNotification(order, err.message);
} finally {
  if (browser) await browser.close();
}
}

// — LOGIN TO FUHRERLOGS —
async function loginToFuhrerlogs(page) {
  await page.goto(CONFIG.FUHRERLOGS_URL, { waitUntil: 'networkidle2', timeout: 30
  await page.waitForTimeout(2000);

  // Click login button
  await page.evaluate(() => {
    const btns = document.querySelectorAll('button');
    for (const btn of btns) {
      if (btn.textContent.includes('Login')) { btn.click(); return; }
    }
  });
  await page.waitForTimeout(1500);

```

```

// Enter email
const emailInput = await page.$('input[type="email"], input[placeholder*="email"]');
if (emailInput) {
    await emailInput.type(CONFIG.FUHRERLOGS_EMAIL, { delay: 50 });
    await page.keyboard.press('Enter');
    await page.waitForTimeout(3000);
}

// Wait for 6-digit code email (poll for 2 minutes)
console.log('[LOGIN] Waiting for 6-digit code email...');
const code = await waitForLoginCode();
console.log(`[LOGIN] Got code: ${code}`);

// Enter code
const codeInput = await page.$('input[maxlength="6"], input[placeholder*="code"]');
if (codeInput) {
    await codeInput.type(code, { delay: 100 });
    await page.keyboard.press('Enter');
    await page.waitForTimeout(3000);
}

console.log('[LOGIN] Login successful!');
}

// — WAIT FOR LOGIN CODE FROM GMAIL —
async function waitForLoginCode(maxWaitMs = 120000) {
    const Imap = require('imap');
    const { simpleParser } = require('mailparser');

    return new Promise((resolve, reject) => {
        const imap = new Imap({
            user: CONFIG.FUHRERLOGS_EMAIL,
            password: CONFIG.GMAIL_PASS,
            host: 'imap.gmail.com',
            port: 993,
            tls: true,
            tlsOptions: { rejectUnauthorized: false },
        });

        const timeout = setTimeout(() => {
            imap.end();
            reject(new Error('Timeout waiting for login code'));
        }, maxWaitMs);

        const startTime = Date.now();

```

```

imap.once('ready', () => {
  imap.openBox('INBOX', false, () => {
    const checkMail = () => {
      imap.search(['UNSEEN', ['FROM', 'fuhrerlogs'], ['SINCE', new Date(start
        if (err || !results || results.length === 0) {
          setTimeout(checkMail, 5000);
          return;
        }

      const fetch = imap.fetch(results, { bodies: '' });
      fetch.on('message', msg => {
        msg.on('body', stream => {
          simpleParser(stream, (err, mail) => {
            if (err) return;
            const text = mail.text || mail.html || '';
            const match = text.match(/\b(\d{6})\b/);
            if (match) {
              clearTimeout(timeout);
              imap.end();
              resolve(match[1]);
            }
          });
        });
      });
    });
    checkMail();
  });
});

imap.once('error', err => { clearTimeout(timeout); reject(err); });
imap.connect();
});
}

// — BUY PRODUCT ON FUHRERLOGS —
async function buyProduct(page, order) {
  const productSlug = PRODUCT_MAP[order.product.toLowerCase()] || '/product/zalan
  await page.goto(CONFIG.FUHRERLOGS_URL + productSlug, { waitUntil: 'networkidle2
  await page.waitForTimeout(2000);

  // Click Purchase button
  await page.evaluate(() => {
    const btns = document.querySelectorAll('button');
    for (const btn of btns) {
      if (btn.textContent.includes('Purchase')) { btn.click(); return; }
    }
  });
}

```

```

});
await page.waitForTimeout(1500);

// Select variant if specified
if (order.variant) {
  await page.evaluate((variantName) => {
    const options = document.querySelectorAll('.variant-option, [class*="varian
    for (const opt of options) {
      if (opt.textContent.includes(variantName)) { opt.click(); return; }
    }
  }, order.variant);
  await page.waitForTimeout(500);
}

// Set quantity
if (order.quantity > 1) {
  for (let i = 1; i < order.quantity; i++) {
    await page.evaluate(() => {
      const plusBtn = document.querySelector('button[onclick*="changeQty"]', but
      if (plusBtn) plusBtn.click();
    });
    await page.waitForTimeout(200);
  }
}

// Click Buy Now
await page.evaluate(() => {
  const btns = document.querySelectorAll('button');
  for (const btn of btns) {
    if (btn.textContent.includes('Buy Now')) { btn.click(); return; }
  }
});
await page.waitForTimeout(2000);

// Use balance to pay
await page.evaluate(() => {
  const balanceOption = document.querySelector('[class*="balance"]', button:cont
  if (balanceOption) balanceOption.click();
});
await page.waitForTimeout(500);

// Click Continue/Checkout
await page.evaluate(() => {
  const btns = document.querySelectorAll('button');
  for (const btn of btns) {
    if (btn.textContent.includes('Continue') || btn.textContent.includes('Check
    btn.click(); return;
  }
});

```

```

    }
  }
});
await page.waitForTimeout(3000);

// Wait for order completion and get content
console.log('[BUY] Waiting for order completion...');
await page.waitForTimeout(5000);

// Click "View Content"
const content = await page.evaluate(() => {
  const btns = document.querySelectorAll('button');
  for (const btn of btns) {
    if (btn.textContent.includes('View Content')) {
      btn.click();
      return null; // will get content after click
    }
  }
});

await page.waitForTimeout(2000);

// Extract content from modal
const productContent = await page.evaluate(() => {
  const modal = document.querySelector('[class*="modal"], [class*="content"]');
  if (modal) return modal.innerText;
  return document.body.innerText.substring(0, 2000);
});

return productContent;
}

// — SEND PRODUCT TO CUSTOMER —
async function sendProductToCustomer(order, content) {
  const mailOptions = {
    from: '"Skaters Shop" <skatedelivery007@gmail.com>',
    to: order.customerEmail,
    subject: 'Your Skaters Order — Product Delivered!',
    html: `
    <div style="font-family:Arial,sans-serif;max-width:600px;margin:0 auto;background-color:#2563eb;padding:24px;text-align:center;">
      <h1 style="margin:0;font-size:24px;color:white;">Skaters</h1>
      <p style="margin:8px 0 0;color:rgba(255,255,255,0.8);">Order Delivered</p>
    </div>
    <div style="padding:32px;">
      <h2 style="color:#2563eb;margin-bottom:8px;">Your product is ready!</h2>
      <p style="color:#7a90b0;">Order ID: <strong style="color:white;">${order

```

```

    <p style="color:#7a90b0;">Product: <strong style="color:white;">${order
    <p style="color:#7a90b0;">Quantity: <strong style="color:white;">${orde

    <div style="background:#090f1e;border:1px solid #1a3a6b;border-radius:1
        <p style="color:#7a90b0;font-size:12px;margin:0 0 10px;text-transform
        <pre style="color:#ffffff;font-family:monospace;font-size:13px;white-
    </div>

    <div style="background:#0a1628;border:1px solid #1a3a6b;border-radius:1
        <p style="color:#7a90b0;font-size:12px;margin:0;">
            ⚠ For any issue with your order, contact our support with proof wi
            <strong style="color:#2563eb;">Skaters Support</strong>
        </p>
    </div>
</div>
<div style="background:#090f1e;padding:16px;text-align:center;border-top:
    <p style="color:#7a90b0;font-size:12px;margin:0;">© 2026 Skaters. All r
</div>
</div>
`
,
};

await transporter.sendMail(mailOptions);
console.log(`[EMAIL] Product sent to ${order.customerEmail}`);
}

// — SEND ERROR NOTIFICATION TO ADMIN —
async function sendErrorNotification(order, errorMsg) {
    await transporter.sendMail({
        from: '"Skaters Server" <skatedelivery007@gmail.com>',
        to: CONFIG.GMAIL_USER,
        subject: `[ERROR] Order ${order.orderId} failed`,
        text: `Order failed!\n\nOrder ID: ${order.orderId}\nCustomer: ${order.custome
    });
}

// — ORDER STATUS —
app.get('/api/order/:orderId', (req, res) => {
    const order = pendingOrders.get(req.params.orderId);
    if (!order) return res.status(404).json({ error: 'Order not found' });
    res.json({ orderId: order.orderId, status: order.status, error: order.error });
});

// — START SERVER —
const PORT = process.env.PORT || 3000;
app.listen(PORT, () => {
    console.log(`Skaters server running on port ${PORT}`);
});

```



```
transporter.verify((err) => {  
  if (err) console.error('[EMAIL] Gmail error:', err.message);  
  else console.log('[EMAIL] Gmail ready to send!');  
});  
});
```